SLIIT UNI
THE KNOWLEDGE UNIVERSITY

Sri Lanka Institute of Information Technology

Faculty of Computing

SE1020 - Object-Oriented Programming

Year 01 and Semester 02

Lecture 02

Constructors

Recall...

- Introduction to OOP.
- Steps in Developing a system using OOP.
- Core Concepts of OOP.
- Encapsulation.
- Abstraction.

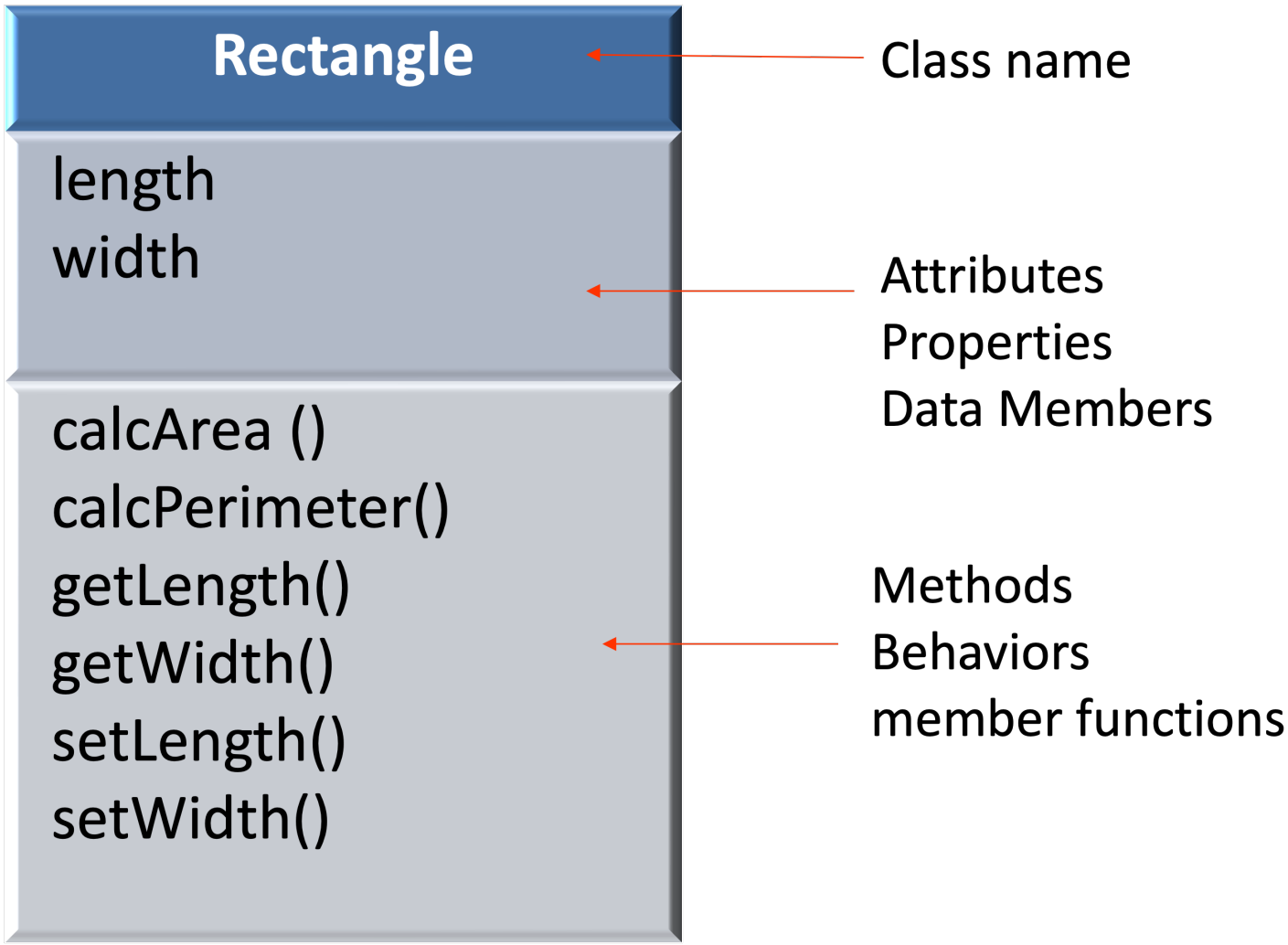
Learning Objectives

- Learn to use encapsulation with private attributes, getters, setters, and constructors in Java.
- Know the difference between default, and overloaded constructors for creating objects.
- Practice method overloading.

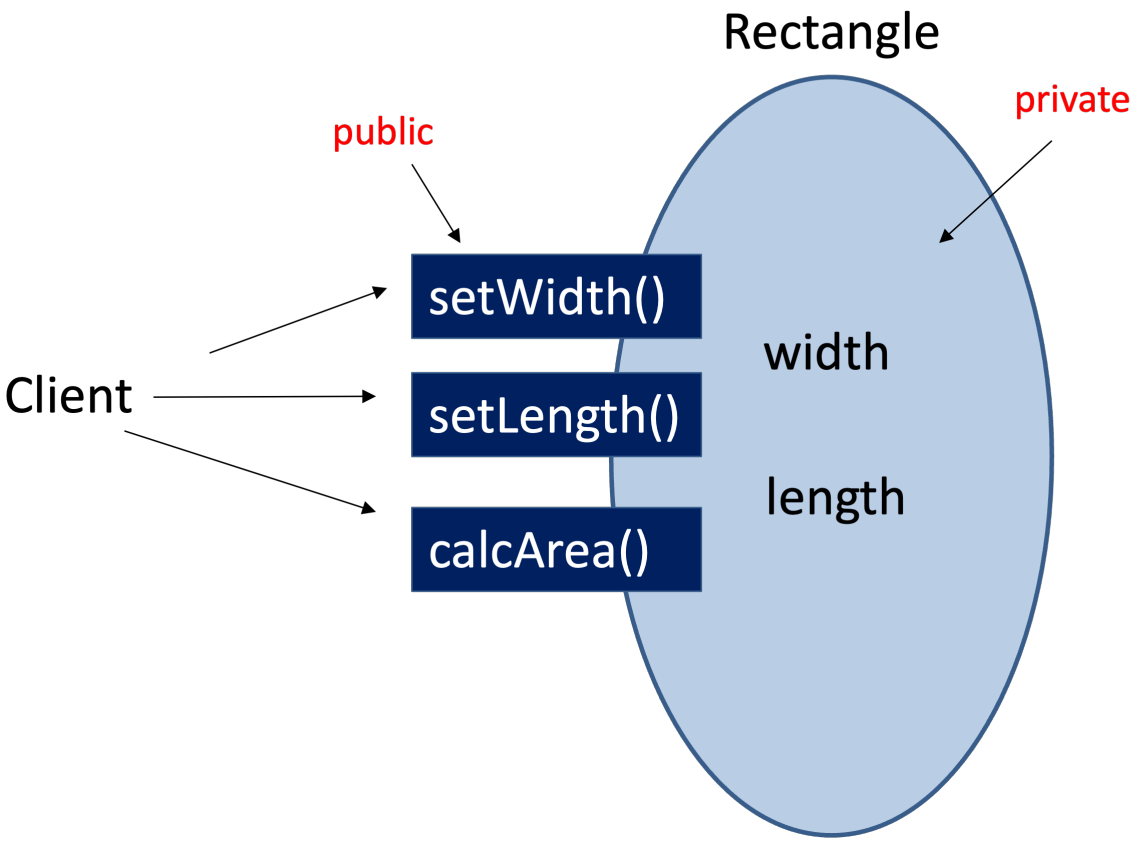
How to write an Object-Oriented Program ?



Rectangle Class



Private & Public



Rectangle Class in Java

```java
public class Rectangle {
    private int length;
    private int width;

    public int getLength() {
        return length;
    }

    public void setLength(int length) {
        this.length = length;
    }

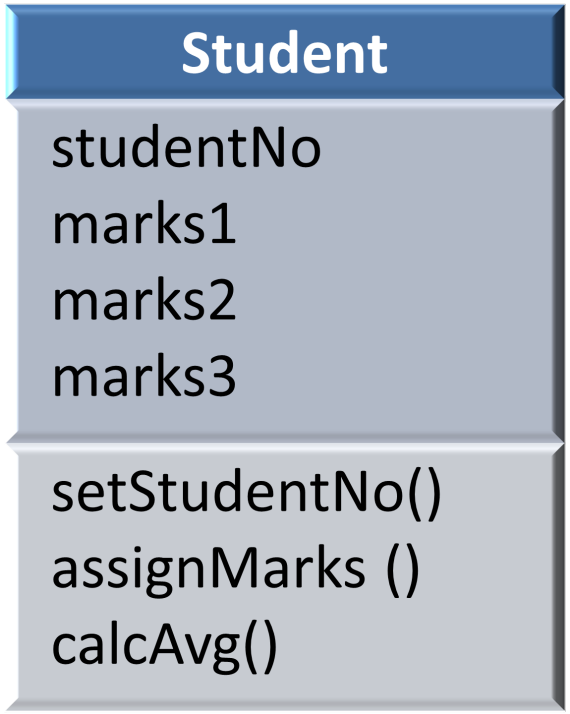
    public int getWidth() {
        return width;
    }

    public void setWidth(int width) {
        this.width = width;
    }

    public int calcArea() {
        return length * width;
    }

    public int calcPerimeter() {
        return 2 * (length + width);
    }
```

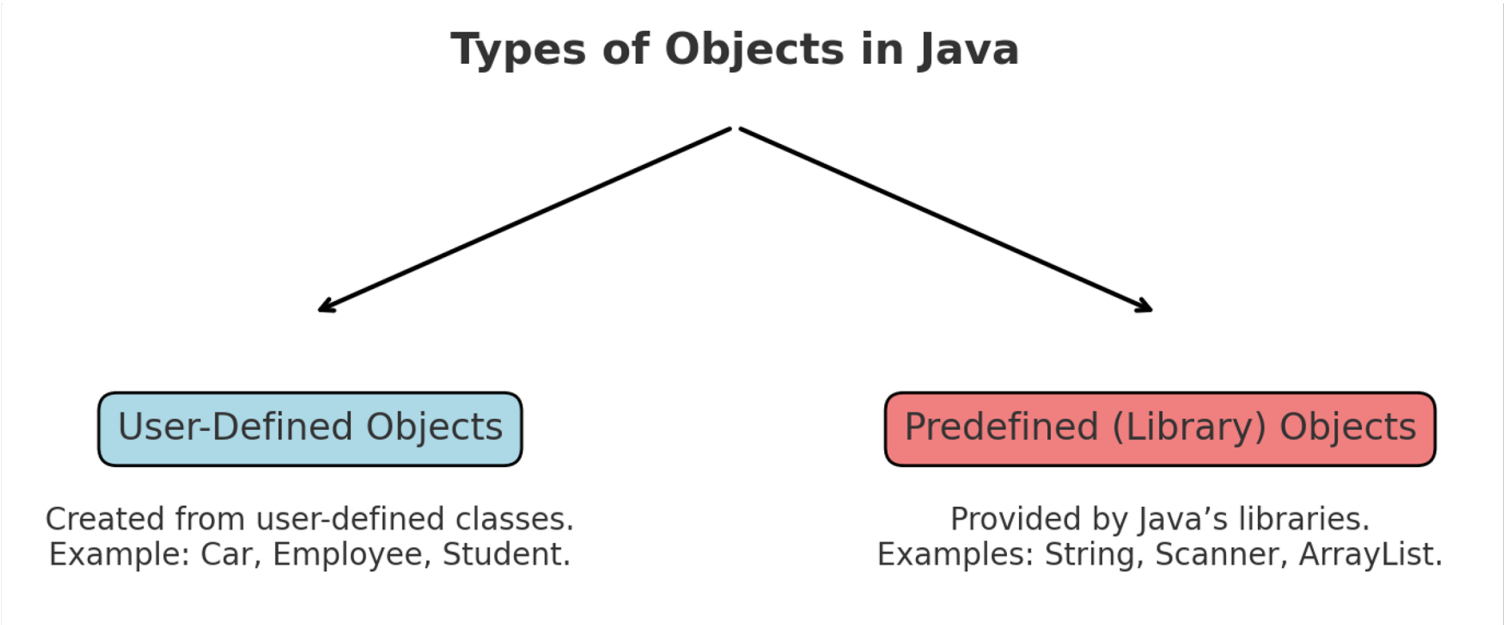
Activity 4 : Implement the Student class in java



What Are Objects in Java?

Java is an Object-Oriented Programming (OOP) language where objects are the main building blocks. An object is an instance of a class, containing:

- Attributes (properties): Representing the state or data
- Methods (functions): Representing the behavior



How to Create Objects in Java?

- Step 01: Define a class

```
public class Rectangle {
    private int length;
    private int width;
}
```

- Step 02: Create an object (dynamic object) using the **new** keyword

```
Rectangle r1 = new Rectangle();
```

- Step 03: Initialize object properties

```
r1.length = 45;
r1.width = 55;
```

How to Use Objects?

- Access object properties

  Use the dot operator (.).

```
System.out.println("The width is " + r1.width);
```

- Call the methods

  Use the dot operator (.).

```
r1.calcArea();
```

```java
public class Rectangle {
    private int length;
    private int width;

    public void calcArea()
    {
        int area = width * length;
        System.out.println("The area is " + area );
    }

    public static void main(String args[])
    {

        Rectangle r1 = new Rectangle();
        r1.length = 45;
        r1.width = 55;

        System.out.println("The width is " + r1.width);

        r1.calcArea();
    }
}
```

Memory Allocation in Java: Object Creation

Sample code:

```java
public class Person {

    private String name;
    private int age;

    public void dispalyDetails() {
        System.out.println("Name : " + name + " , Age : " + age);
    }

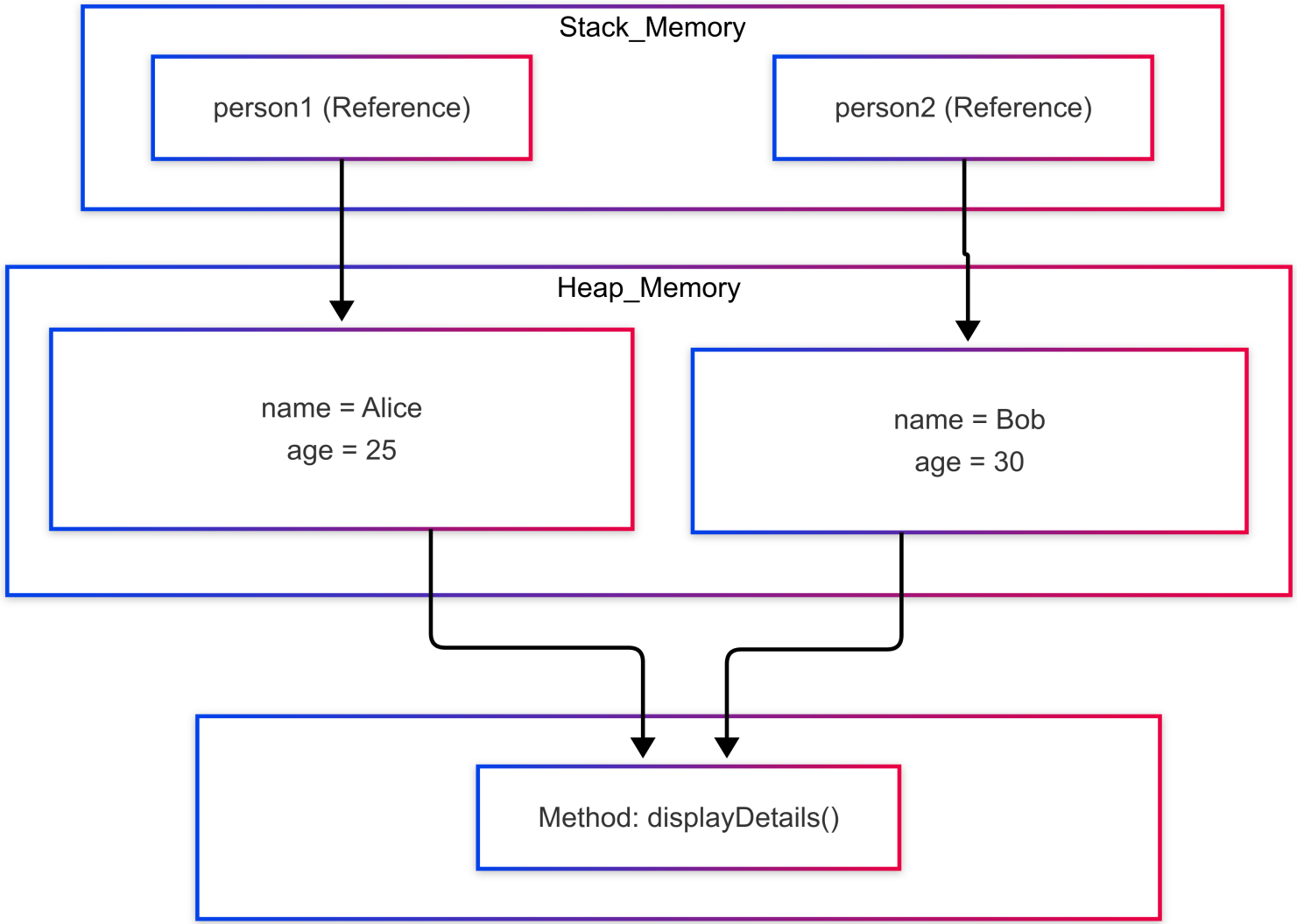
    public static void main(String[] args) {
        //Object Creation
        Person person1 = new Person();

        //Allocating values to attributes
        person1.name = "Alice";
        person1.age = 25;

        //Calling methods
        person1.dispalyDetails();

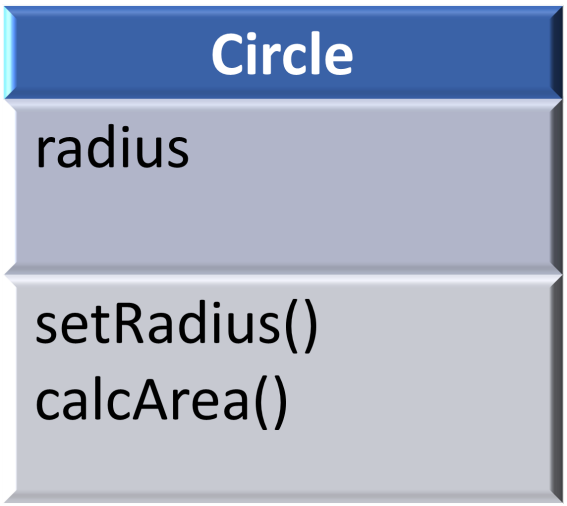
        Person person2 = new Person();
        person2.name = "Bob";
        person2.age = 30;
        person2.dispalyDetails();

    }
```


Stack_Memory
person1 (Reference)
person2 (Reference)
Heap_Memory
name = Alice
age = 25
name = Bob
age = 30
Method: displayDetails()

Activity 05

- Implement the **Circle** class (refer to the given class diagram) with necessary implementation for the methods.
- Create a main method and inside that create 3 objects of Circle class. Then calculate and print the areas separately.



Access Modifiers

Access Modifier	Within Class	Within Package	Outside Package (Subclass)	Outside Package (Other Classes)
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
default	Yes	Yes	No	No
private	Yes	No	No	No

- **public** → Accessible from **anywhere**.
- **protected** → Accessible **within the same package** and **in subclasses outside the package.**
- **default** → Accessible **only within the same package.**
- **private** → Accessible **only within the same class.**

Getters and Setters in java

All properties of a class should have restricted access (declared as private) to ensure encapsulation and data security. These properties can only be accessed or modified from outside the class through public methods.

Refer to the given scenario.

```java
class Rectangle {
    private int length;
    private int width;

    public int calcArea() {
        return length * width;
    }

    public int calcPerimeter() {
        return 2 * (length + width);
    }
}


public class RectangleApp {

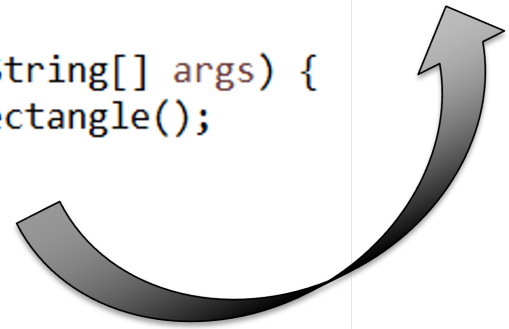
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle();
        r1.length = 15;
        r1.width = 35;

    }
}
```

Since the properties length and width are private, those cannot be accessed from the main function of another class.

In this situation we usually write two methods per property.

A set method (setters) and a get method (getters).



Setters (Mutators)

- Setters are public methods used to set or update the values of private attributes in a class. They provide controlled access to the properties of an object.
- By using setters, you can include validation or additional logic before assigning a value to a property.

```
public void setLength(int len) {
    if (len > 0) {
        this.length = len;
    } else {
        System.out.println("Length must be positive.");
    }
}
```

Getters (Accessors)

- Getters are public methods used to retrieve the values of private properties in a class.
- Getters ensure that private properties cannot be accessed directly from outside the class but can be read through these methods.

```
// Getter method
    public int getLength() {
        return length;
    }
```

Sample code:

```java
class Rectangle {
    private int length;
    private int width;

    public void setLength(int len) {
        if (length > 0) {
            this.length = len;
        } else {
            System.out.println("Length must be positive.");
        }
    }

    public int getLength() {
        return length;
    }

    public void setWidth(int width) {
        this.width = width;
    }

    public int getWidth() {
        return width;
    }
}
```

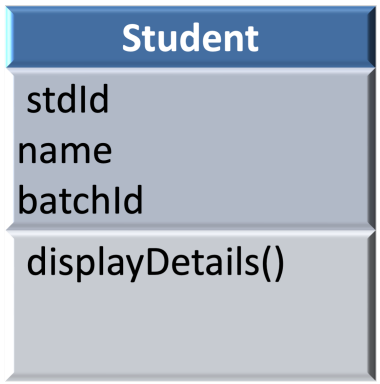
```java
public class RectangleApp {

    public static void main(String[] args) {
        Rectangle r1 = new Rectangle();
        r1.setLength(15);
        r1.setWidth(25);

        System.out.println("The length is " + r1.getLength());
        System.out.println("The width is " + r1.getWidth());
    }
}
```

Activity 06

- Implement the given class diagram using java. Implement getters and setters for the 3 attributes.
  [Hint: **stdId** should be always greater than 0]
- Create another class called **StudentApp** with the main method. Create 2 Student objects and accept the user inputs for the 3 attributes and assign them.
- Display the name and the student id of each student.



Method Overloading

- Method overloading allows multiple methods in the **same class** to have the same name but **different parameter lists**. In here;
  - Methods must have the same name.
  - Methods must differ in number, type, or order of parameters.
  - Return type of a method does not affect method overloading.

```java
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println("Sum of two integers: " + calc.add(10, 20));
        System.out.println("Sum of three integers: " + calc.add(10, 20, 30));
        System.out.println("Sum of two doubles: " + calc.add(10.5, 20.5));
    }
}
```

Employee Class

```java
public class Employee {

    private String name;
    private int id;
    private String department;
    private double salary;

    public void displayDetails() {
        System.out.println("======================");
        System.out.println("Employee Details:");
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
        System.out.println("Department: " + department);
        System.out.println("Salary: " + salary);
        System.out.println("======================");
    }
}
```

How do we assign values to the emp object when it is created?

The output shown below displays random values

Note : We have not used setters and getters for simplicity

```java
public class EmployeeApp {
    public static void main(String[] args) {

        Employee emp = new Employee();
        emp.displayDetails();
    }
}
```

Output:

```
======================
Employee Details:
Name: null
ID: 0
Department: null
Salary: 0.0
======================
```

```java
public class Employee {

    private String name;
    private int id;
    private String department;
    private double salary;

    public void assignDetails() {
        this.name = "testName";
        this.id = 10;
        this.department = "testDept";
        this.salary = 10000.00;
    }

    public void assignDetails(String pName, int pId) {
        this.name = pName;
        this.id = pId;
    }

    public void displayDetails() {
        System.out.println("====================");
        System.out.println("Employee Details:");
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
        System.out.println("Department: " + department);
        System.out.println("Salary: " + salary);
        System.out.println("====================");
    }
}
```

```
====================
Employee Details:
Name: testName
ID: 10
Department: testDept
Salary: 10000.0
====================
```

Here we can assign the values to the attributes by calling assignDetails() and overloaded assignDetails() methods. But still we have to explicitly call those methods after the object creation.

But what if we want to do this automatically when an object is created?

```java
public class EmployeeApp {
    public static void main(String[] args) {

        Employee emp = new Employee();
        emp.assignDetails();

        emp.displayDetails();
    }
}
```

Initializing the Attributes of an Object

- Initialize means setting values to variables (attributes) at the beginning (usually when they are declared).
- Although we add another method to the class called **"assignDetails()"**, it does not initialize the attributes automatically, instead when called it assigns or overwrites the garbage values that are already in the attributes.
- We could automatically initialize attributes in a class by using a special kind of methods called "Constructors"

Constructors

- A constructor in Java is a special method used to initialize objects.
- It is called automatically when an object is created and helps assign initial values to the fields of the class.
  A constructor:
  - Shares the same name as the class.
  - Does not have a return type, not even **void**.
- There can be a **default constructors** or **constructors with parameters**
- When an object is declared the appropriate constructor is executed.

Default Constructors

- A default constructor is a constructor that does not take any arguments.
- Can be used to initialized attributes to default values
- If you do not explicitly define any constructor in your class, Java automatically provides a default constructor.
- This constructor initializes the object with default values:
  - For objects: null
  - For integers: 0
  - For floating-point numbers: 0.0

This is how we can add a default constructor to the Employee class

```java
public class Employee {

    private String name;
    private int id;
    private String department;
    private double salary;

    public Employee() {
        this.name = "testName";
        this.id = 10;
        this.department = "testDept";
        this.salary = 10000.00;
    }

    public void displayDetails() {
        System.out.println("====================");
        System.out.println("Employee Details:");
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
        System.out.println("Department: " + department);
        System.out.println("Salary: " + salary);
        System.out.println("====================");
    }
}
```

```java
public class EmployeeApp {
    public static void main(String[] args) {

        Employee emp = new Employee();
        emp.displayDetails();
    }
}
```

When we create the emp object in the main method, the default constructor is automatically executed.

```
====================
Employee Details:
Name: testName
ID: 10
Department: testDept
Salary: 10000.0
====================
```

Overloaded Constructors (Constructors with Parameters)

- Overloaded constructors allow a class to have multiple constructors with the same name but different parameter lists.
- Each overloaded constructor performs a specific initialization based on the arguments provided.
- Overloaded constructors are useful when you need to provide multiple ways to initialize an object.

This is how we can add overloaded constructors to the Employee class

```
public class Employee {

    private String name;
    private int id;
    private String department;
    private double salary;

    public Employee() {
        this.name = "testName";
        this.id = 10;
        this.department = "testDept";
        this.salary = 10000.00;
    }

    public Employee(String pname, int pid, String pdepartment, double psalary) {
        this.name = pname;
        this.id = pid;
        this.department = pdepartment;
        this.salary = psalary;
    }

    public Employee(String pname, int pid) {
        this.name = pname;
        this.id = pid;
    }

    public void displayDetails() {
        ...
    }
}
```

```java
public class EmployeeApp {
    public static void main(String[] args) {

        Employee emp1 = new Employee();
        emp1.displayDetails();

        Employee emp2 = new Employee("Kamal", 101, "IT", 200000.00);
        emp2.displayDetails();

        Employee emp3 = new Employee("Sunil", 201);
        emp3.displayDetails();
    }
}
```

When we create the emp object in the main() program,
the particular overloaded constructor is automatically executed.

```
======================
Employee Details:
Name: testName
ID: 10
Department: testDept
Salary: 10000.0
======================

======================
Employee Details:
Name: Kamal
ID: 101
Department: IT
Salary: 200000.0
======================

======================
Employee Details:
Name: Sunil
ID: 201
Department: null
Salary: 0.0
======================
```

Activity 07

1. Create a class called **Product** with attributes: **productId**, **name**, **price**, and **quantity**.
2. Implement the following:
   i. A default constructor that initializes attributes with default values.
   ii. A parameterized constructor to initialize all attributes.
   iii. Getters and setters for each attribute, including validation (e.g., price should be greater than 0).
   iv. An **assignDetails()** method that accepts input to set attributes.
   v. A **displayDetails()** method to print all product information.
3. Create another class called **ProductApp** class:
   i. Create an object using the default constructor and display its details.
   ii. Create another object using the parameterized constructor and display its details.
   iii. Use the assignDetails method to update an object and display the updated details.

Summary

- Learn to implement **encapsulation** using private attributes, getters, setters, and constructors in Java.
- Differentiate between **default constructors, parameterized constructors,** and **overloaded constructors** for initializing objects.
- Gain hands-onexperience with **method overloading** and its use in enhancing code flexibility and readability.

Thank You!